

DSA – Java

1. Two Sum:

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to `target`*.

You may assume that each input would have *exactly one solution*, and you may not use the *same* element twice.

Soln:

brute force approach is to consider every pair of elements and check if their sum equals the target. This can be done using nested loops, where the outer loop iterates from the first element to the second-to-last element, and the inner loop iterates from the next element to the last element. However, this approach has a time complexity of $O(n^2)$.

Hash Map

To achieve a faster solution, we can trade a bit of memory for a significant speed boost. The idea is to iterate through the array a single time, using a hash map to keep track of the numbers we've seen so far and their indices.

For each element in the array, we do the following:

1. Calculate the complement we need to find (i.e., `complement = target - current_number`).
2. Check if this complement already exists as a key in our hash map.
 - If it exists, we've found our solution! The value associated with the complement's key is its index, and we have the index of our current number. We can return these two indices immediately.
 - If it does not exist, we add the current number and its index to the hash map. This prepares us for future iterations, in case the current number is the complement for a number we haven't seen yet.

This one-pass strategy is extremely efficient because the hash map lookup is so fast.

4. Median of Two Sorted Arrays (Merge Approach)

Problem Statement

Given two sorted integer arrays `nums1` and `nums2`, merge them into a single sorted array and return the median of the merged array.

Approach

Since both arrays are already sorted, we can merge them using the **two-pointer technique**, similar to the **Merge step of Merge Sort**. Once the merged array is obtained, the median can be found by checking whether the total number of elements is odd or even.

Algorithm

Step 1: Initialize Variables

- Let `m` be the length of `nums1`.
 - Let `n` be the length of `nums2`.
 - Create a new array `merged` of size `m + n`.
 - Initialize three pointers:
 - `i` → points to the current element in `nums1`.
 - `j` → points to the current element in `nums2`.
 - `k` → points to the current position in the merged array.
-

Step 2: Merge the Arrays

Compare the current elements of both arrays.

- If `nums1[i] <= nums2[j]`, copy `nums1[i]` into `merged[k]` and increment `i` and `k`.
- Otherwise, copy `nums2[j]` into `merged[k]` and increment `j` and `k`.

Repeat this process until one of the arrays is completely traversed.

Step 3: Copy Remaining Elements

If elements are still left in either array, copy them directly into the merged array.

- Copy all remaining elements from `nums1` (if any).

- Copy all remaining elements from `nums2` (if any).

At this point, the `merged` array contains all elements in sorted order.

Step 4: Find the Median

Let

`mid = merged.length / 2`

Case 1: Total number of elements is Odd

The median is the middle element.

Formula:

`Median = merged[mid]`

Case 2: Total number of elements is Even

The median is the average of the two middle elements.

Formula:

`Median = (merged[mid - 1] + merged[mid]) / 2.0`

The division is performed using `2.0` to ensure the result is of type `double`.

Two Sum II – Input Array Is Sorted (Two-Pointer Approach)

Problem Statement

Given a **sorted** array of integers `numbers` and an integer `target`, find two numbers such that they add up to the target.

Return the **1-based indices** of the two numbers as an array `[index1, index2]`.

It is guaranteed that exactly one solution exists, and the same element cannot be used twice.

Approach

Since the array is already sorted in non-decreasing order, we can use the **Two-Pointer Technique** instead of a HashMap.

We place:

- One pointer (`i`) at the beginning of the array.
- Another pointer (`j`) at the end of the array.

Based on the sum of the two elements, we decide which pointer to move.

Key Observation

Because the array is sorted:

- If the current sum is **greater than the target**, we need a smaller sum.
 - Move the right pointer one step to the left.
- If the current sum is **less than the target**, we need a larger sum.
 - Move the left pointer one step to the right.
- If the current sum equals the target, we have found the answer.

This allows us to eliminate unnecessary elements efficiently.

Reverse String (Two-Pointer Approach)

Problem Statement

Given a character array `S`, reverse the array **in-place** without using any extra array.

Approach

The idea is to use the **Two-Pointer Technique**.

- Place one pointer (`i`) at the beginning of the array.
- Place another pointer (`j`) at the end of the array.
- Swap the characters at both pointers.
- Move the left pointer one step forward and the right pointer one step backward.
- Continue this process until both pointers meet or cross each other.

Since the reversal is performed by swapping elements within the same array, no additional space is required.

Valid Palindrome (Two-Pointer Approach)

Problem Statement

Given a string `S`, determine whether it is a palindrome after:

- Converting all uppercase letters to lowercase.
- Removing all non-alphanumeric characters.

Return `true` if the resulting string is a palindrome; otherwise, return `false`.

Approach

A palindrome is a string that reads the same forward and backward.

The given string may contain spaces, punctuation marks, and special characters. Therefore, we first preprocess the string by removing all non-alphanumeric characters and converting it to lowercase. Then, we use the **Two-Pointer Technique** to compare characters from both ends of the string.

```
s = s.replaceAll("[^a-zA-Z0-9]", "");
```

```
s = s.toLowerCase();
```

Move Zeroes (Two-Pointer Approach)

Problem Statement

Given an integer array `nums`, move all the 0s to the end of the array while maintaining the relative order of the non-zero elements. The operation must be performed **in-place**, without creating another array.

Approach

The idea is to use the **Two-Pointer Technique**.

- The **left pointer** keeps track of the position where the next non-zero element should be placed.
- The **right pointer** traverses the entire array from left to right.

Whenever a non-zero element is encountered, it is swapped with the element at the `left` pointer. After the swap, the `left` pointer is incremented, indicating that the next position is now available for another non-zero element.

This ensures that:

- All non-zero elements are moved to the beginning of the array.
 - All zeroes automatically shift to the end.
 - The relative order of non-zero elements remains unchanged.
-

Algorithm

Step 1: Initialize Pointer

Initialize a pointer `left` to 0.

- `left` represents the position where the next non-zero element should be placed.
-

Step 2: Traverse the Array

Use another pointer `right` to traverse the array from index 0 to $n - 1$.

For every element:

- If `nums[right]` is **non-zero**:
 - Swap `nums[left]` and `nums[right]`.
 - Increment `left`.

- If `nums[right]` is **zero**:
 - Do nothing.
 - Simply move to the next element.
-

Step 3: Finish Traversal

Once the traversal is complete:

- All non-zero elements occupy the beginning of the array.
 - All zeroes are automatically moved to the end.
-

Remove Duplicates from Sorted Array (Two-Pointer Approach)

Problem Statement

Given a **sorted** integer array `nums`, remove the duplicate elements **in-place** such that each unique element appears only once. Return the number of unique elements (k).

The relative order of the elements must remain the same, and no extra array should be used.

Approach

Since the array is already sorted, all duplicate elements appear **consecutively**. This property allows us to use the **Two-Pointer Technique**.

We maintain two pointers:

- **i (Slow Pointer)**: Points to the last unique element in the array.
- **j (Fast Pointer)**: Traverses the array to find new unique elements.

Whenever a new unique element is found, it is placed immediately after the last unique element.

Algorithm

Step 1: Initialize Pointers

- Set $i = 0$.

- Set $j = 0$.

Initially, the first element is always unique.

Step 2: Traverse the Array

Move the fast pointer j from the beginning to the end of the array.

For every element:

- Compare `nums[j]` with `nums[i]`.
 - If both are equal, the element is a duplicate, so simply move j forward.
 - If they are different, a new unique element has been found.
 - Increment i .
 - Copy `nums[j]` to `nums[i]`.
 - Continue scanning the remaining elements.
-

Step 3: Return the Number of Unique Elements

After the traversal is complete:

- Indices 0 to i contain all the unique elements.
- Therefore, the total number of unique elements is:

$i + 1$

Remove Element (Two-Pointer Approach)

Problem Statement

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` **in-place** and return the number of remaining elements.

The relative order of the remaining elements may be preserved. The elements beyond the returned length are not important.

Approach

The idea is to use the **Two-Pointer Technique**.

- Traverse the array using one pointer (i).
- Maintain another pointer (`index`) that keeps track of the position where the next valid element should be placed.

- Whenever an element is **not equal** to `val`, copy it to `nums[index]` and increment `index`.
- Ignore all occurrences of `val`.

After the traversal, the first `index` elements of the array contain all the required elements.

Algorithm

Step 1: Initialize a Pointer

Create a variable `index` and initialize it to 0.

`index` represents the position where the next valid element will be stored.

Step 2: Traverse the Array

Loop through every element in the array.

For each element:

- If `nums[i] != val`
 - Copy the element to `nums[index]`.
 - Increment `index`.
 - Otherwise, skip the element.
-

Step 3: Return the Result

After the traversal is complete, `index` represents the number of elements remaining after removing all occurrences of `val`.

Return `index`.

Squares of a Sorted Array (Two-Pointer Approach)

Problem Statement

Given a sorted integer array `nums` in non-decreasing order, return an array containing the squares of each number, also sorted in non-decreasing order.

Intuition

Although the input array is sorted, squaring each element may change the order because negative numbers become positive after squaring.

Example:

```
nums = [-7, -3, 2, 3, 11]
```

```
After squaring:  
[49, 9, 4, 9, 121]
```

The squared values are **not** sorted.

Observe that the **largest square** always comes from one of the two ends of the array:

- The left end may contain a large negative number.
- The right end contains the largest positive number.

Therefore, compare the absolute values at both ends and place the larger square at the end of the result array.

Algorithm

Step 1: Initialize Pointers

Use three pointers:

- `i` → Points to the beginning of the array.
- `j` → Points to the end of the array.
- `k` → Points to the last index of the result array.

Create a new array `result` of the same size as `nums`.

Step 2: Compare Absolute Values

While `i <= j`:

- Compare `|nums[i]|` and `|nums[j]|`.
- If `|nums[i]| > |nums[j]|`
 - Square `nums[i]`.
 - Store it at `result[k]`.
 - Increment `i`.
- Otherwise
 - Square `nums[j]`.

- Store it at `result[k]`.
- Decrement `j`.

After inserting an element, decrement `k`.

Step 3: Return the Result

When the loop ends, every position of the result array contains the squared values in sorted order.

3Sum Problem (Sorting + Two Pointers)

Problem Statement

Given an integer array `nums`, return **all unique triplets** `[nums[i], nums[j], nums[k]]` such that:

- $i \neq j \neq k$
- $nums[i] + nums[j] + nums[k] = 0$

The solution should not contain duplicate triplets.

Approach

The brute force approach checks every possible triplet, resulting in $O(n^3)$ time complexity. This is inefficient for large arrays.

A better approach is to:

1. Sort the array.
2. Fix one element.
3. Use the **Two Pointer Technique** to find the remaining two elements whose sum equals the negative of the fixed element.

Sorting allows us to efficiently move pointers based on the current sum and also makes it easy to skip duplicate elements.

Algorithm

Step 1: Sort the Array

Sort the given array in ascending order.

```
Arrays.sort(nums);
```

Sorting is necessary because:

- It enables the two-pointer technique.
 - Duplicate triplets can be removed efficiently.
-

Step 2: Fix the First Element

Traverse the array using index `i`.

For every iteration,

- Consider `nums[i]` as the first element of the triplet.
- Search for the remaining two numbers.

Skip duplicate values of `nums[i]` to avoid duplicate triplets.

```
if(i > 0 && nums[i] == nums[i - 1])  
    continue;
```

Step 3: Initialize Two Pointers

For every fixed element,

- `j = i + 1` (left pointer)
- `k = nums.length - 1` (right pointer)

These pointers search the remaining portion of the array.

Step 4: Calculate the Sum

Compute

```
sum = nums[i] + nums[j] + nums[k]
```

There are three possibilities.

Case 1: Sum > 0

The sum is too large.

Move the right pointer one step left.

```
k--
```

Since the array is sorted, moving left decreases the sum.

Case 2: Sum < 0

The sum is too small.

Move the left pointer one step right.

`j++`

Moving right increases the sum.

Case 3: Sum == 0

A valid triplet has been found.

Store the triplet in the answer list.

```
list.add(Arrays.asList(nums[i], nums[j], nums[k]));
```

Move both pointers.

```
j++  
k--
```

Skip duplicate values for both pointers.

```
while(j < k && nums[j] == nums[j - 1])  
    j++;
```

```
while(j < k && nums[k] == nums[k + 1])  
    k--;
```

This ensures that only unique triplets are added.

Container With Most Water (Two Pointer Approach)

Problem Statement

Given an integer array `height`, where each element represents the height of a vertical line, find two lines that together with the x-axis form a container such that the container holds the maximum amount of water.

Return the maximum amount of water that can be stored.

Approach

The brute-force solution checks every possible pair of lines, resulting in **O(n²)** time complexity.

A more efficient solution uses the **Two Pointer Technique**, achieving **O(n)** time complexity.

The idea is to start with one pointer at the beginning of the array and another at the end. These two pointers represent the widest possible container. At each step, calculate the area and move the pointer corresponding to the shorter line.

Observation

The amount of water that can be stored depends on two factors:

- Width between the two lines
- Height of the shorter line

Formula

$\text{Area} = \min(\text{height}[\text{left}], \text{height}[\text{right}]) \times (\text{right} - \text{left})$

Since the shorter line limits the water level, increasing the height of the shorter line is the only way to potentially obtain a larger area.

Algorithm

Step 1: Initialize Two Pointers

- `left = 0`
- `right = height.length - 1`
- `maxArea = 0`

Initially, consider the widest possible container.

Step 2: Calculate Current Area

Find the smaller of the two heights.

`length = min(height[left], height[right])`

Calculate the width.

`width = right - left`

Calculate the area.

`area = length × width`

Update the maximum area if the current area is larger.

Step 3: Move the Smaller Height Pointer

If

```
height[left] < height[right]
```

move the left pointer one step to the right.

Otherwise,

move the right pointer one step to the left.

This is because moving the taller line cannot increase the height of the container while the width decreases.

Step 4: Repeat

Continue until

```
left < right
```

The maximum area found during the process is the answer.

Dry Run

Example

Input

```
height = [1,8,6,2,5,4,8,3,7]
```

Left	Right	Heights	Width	Area	Maximum Area
0	8	(1,7)	8	8	8
1	8	(8,7)	7	49	49
1	7	(8,3)	6	18	49
1	6	(8,8)	5	40	49
2	6	(6,8)	4	24	49
3	6	(2,8)	3	6	49
4	6	(5,8)	2	10	49
5	6	(4,8)	1	4	49

Pointers meet, so the algorithm stops.

Output

49

Why Do We Move the Smaller Pointer?

Suppose

Left Height = 3
Right Height = 8
Width = 6

Area = $3 \times 6 = 18$

If we move the taller pointer:

Width decreases
Minimum height is still at most 3
Area cannot increase.

If we move the shorter pointer:

Width decreases,
but we may find a taller line,
which can increase the minimum height and produce a larger area.

Therefore, always move the pointer with the **smaller height**.

Correctness

- The shorter line always determines the water level.
 - Moving the taller line cannot increase the water level because the shorter line remains the limiting factor.
 - By moving only the shorter pointer, every possible better candidate is considered.
 - Thus, the algorithm guarantees finding the maximum possible container area.
-

Complexity Analysis

Time Complexity: $O(n)$

- Each pointer moves at most n times.
- Every element is visited at most once.

Space Complexity: $O(1)$

- Only a few variables are used.
 - No extra data structures are required.
-

Key Points

- Uses the **Two Pointer Technique**.
 - Starts with the maximum possible width.
 - At every step:
 - Compute the current area.
 - Update the maximum area.
 - Move the pointer with the smaller height.
 - Runs in **$O(n)$** time and **$O(1)$** extra space.
 - Much more efficient than the brute-force approach ($O(n^2)$).
-

Formula Summary

Width = right - left

Height = $\min(\text{height}[\text{left}], \text{height}[\text{right}])$

Area = Width $\times$ Height

Maximum Area = $\max(\text{Maximum Area}, \text{Current Area})$

3Sum Closest (Two Pointer Approach)

Problem Statement

Given an integer array `nums` of length `n` and an integer `target`, find three integers in `nums` such that the sum is closest to the target.

Return the sum of the three integers.

It is guaranteed that exactly one solution exists.

Approach

The brute-force approach would be to check every possible triplet, which takes **$O(n^3)$** time.

A more efficient approach is to first **sort the array** and then use the **Two Pointer Technique**.

Sorting allows us to efficiently increase or decrease the sum by moving pointers instead of checking every triplet.

Algorithm

Step 1: Sort the Array

Sort the array in ascending order.

After sorting,

- Smaller elements are on the left.
- Larger elements are on the right.

This property allows the use of two pointers.

Step 2: Initialize the Closest Difference

Maintain a variable

`orgdiff`

to store the smallest difference found so far.

Initially, assign it a very large value.

`orgdiff = Integer.MAX_VALUE`

The actual closest sum at any point is

`Closest Sum = target + orgdiff`

Step 3: Fix the First Element

Traverse the array from index 0 to $n - 3$.

Treat the current element as the first element of the triplet.

for $i = 0$ to $n-3$

Skip duplicate values because choosing the same first element again produces identical triplets.

Step 4: Apply Two Pointers

For every fixed element,

- $j = i + 1$
- $k = n - 1$

Now calculate

```
sum = nums[i] + nums[j] + nums[k]
```

and

```
diff = sum - target
```

Step 5: Update the Closest Sum

If

```
abs(diff) < abs(orgdiff)
```

then a better (closer) answer has been found.

Update

```
orgdiff = diff
```

Step 6: Move the Pointers

There are three possibilities.

Case 1

```
sum == target
```

This is the best possible answer because the difference is zero.

Return the sum immediately.

Case 2

```
sum > target
```

The current sum is too large.

Decrease the sum by moving the right pointer left.

```
k--
```

Case 3

```
sum < target
```

The current sum is too small.

Increase the sum by moving the left pointer right.

```
j++
```

Repeat until

$j \geq k$

Step 7: Return the Result

The closest difference is stored in

orgdiff

Hence,

Closest Sum = target + orgdiff

Dry Run

Input

nums = [-1, 2, 1, -4]

target = 1

Step 1

Sort the array

[-4, -1, 1, 2]

Iteration 1

Fixed element

$i = 0$

nums[i] = -4

Pointers

$j = 1$

$k = 3$

Triplet

$-4 + (-1) + 2 = -3$

Difference

$-3 - 1 = -4$

Update

`orgdiff = -4`

Since

`sum < target`

Move

`j++`

Now

`j = 2`
`k = 3`

Triplet

`-4 + 1 + 2 = -1`

Difference

`-1 - 1 = -2`

Since

`|-2| < |-4|`

Update

`orgdiff = -2`

Move

`j++`

Pointers cross.

Iteration 2

Fixed element

`i = 1`

`nums[i] = -1`

Pointers

`j = 2`
`k = 3`

Triplet

`-1 + 1 + 2 = 2`

Difference

$$2 - 1 = 1$$

Since

$$|1| < |-2|$$

Update

$$\text{orgdiff} = 1$$

Current sum is greater than target.

Move

$$k--$$

Pointers cross.

Return

$$\text{target} + \text{orgdiff}$$

$$= 1 + 1$$

$$= 2$$

Correctness

- Sorting enables the use of two pointers.
 - For every fixed first element, the algorithm checks all possible pairs using the left and right pointers.
 - Whenever a triplet with a smaller absolute difference is found, it updates the answer.
 - Moving the pointers ensures that every relevant combination is considered without unnecessary comparisons.
 - Therefore, the algorithm always returns the triplet sum closest to the target.
-

Complexity Analysis

Time Complexity: $O(n^2)$

- Sorting takes $O(n \log n)$.
- The outer loop runs n times.
- The two pointers traverse the array once for each fixed element.

Overall,

$$O(n \log n + n^2) = O(n^2)$$

Space Complexity: $O(1)$

- Only a few variables are used.
 - Apart from sorting (which may use implementation-dependent stack space), no additional data structures are required.
-

Key Points

- Sort the array before applying the two-pointer technique.
- Fix one element and search for the remaining two using two pointers.
- Maintain the smallest difference instead of directly storing the sum.
- If the difference becomes zero, return immediately because an exact match is the closest possible answer.
- This approach is significantly more efficient than the brute-force $O(n^3)$ solution.

4Sum (Two Pointers + Sorting)

Problem Statement

Given an integer array `nums` and an integer `target`, return **all unique quadruplets** `[nums[a], nums[b], nums[c], nums[d]]` such that:

- `a`, `b`, `c`, and `d` are distinct indices.
- `nums[a] + nums[b] + nums[c] + nums[d] = target`.

The solution set must not contain duplicate quadruplets.

Approach

The idea is an extension of the **3Sum** problem.

1. **Sort** the array.
2. Fix the **first** element using the first loop.
3. Fix the **second** element using the second loop.

4. Use the **Two Pointer Technique** to find the remaining two elements whose sum equals the remaining target.
 5. Skip duplicate values at every stage to avoid duplicate quadruplets.
-

Algorithm

Step 1: Sort the Array

Sorting helps in:

- Applying the two-pointer technique.
 - Easily skipping duplicate elements.
 - Producing quadruplets in sorted order.
-

Step 2: Fix the First Element

Traverse the array using index i .

$i = 0$ to $n - 4$

Skip duplicate values.

If

`nums[i] == nums[i - 1]`

continue to the next iteration.

This ensures that the same first element is not processed multiple times.

Step 3: Fix the Second Element

For every i , traverse using another pointer j .

$j = i + 1$ to $n - 3$

Again, skip duplicate values.

If

`nums[j] == nums[j - 1]`

continue.

Now two numbers are fixed:

`nums[i]`
`nums[j]`

Step 4: Apply Two Pointers

Initialize:

```
k = j + 1  
l = n - 1
```

Now calculate

```
sum = nums[i] + nums[j] + nums[k] + nums[l]
```

Three cases are possible:

Case 1

If

```
sum < target
```

Move the left pointer forward.

```
k++
```

because increasing k increases the total sum.

Case 2

If

```
sum > target
```

Move the right pointer backward.

```
l--
```

because decreasing l decreases the total sum.

Case 3

If

```
sum == target
```

A valid quadruplet is found.

Add it to the answer list.

Then

```
k++  
l--
```

Skip duplicate values of both pointers to avoid repeated quadruplets.

Why is long used?

The statement

```
long sum = (long) nums[i] + nums[j] + nums[k] + nums[l];
```

prevents **integer overflow**.

Example:

```
nums = [10000000000,  
        10000000000,  
        10000000000,  
        10000000000]
```

Their sum exceeds the maximum value of `int`.

Using `long` ensures the calculation remains correct.

Dry Run

Input

```
nums = [1, 0, -1, 0, -2, 2]  
target = 0
```

Step 1

After sorting

```
[-2, -1, 0, 0, 1, 2]
```

Step 2

Choose

```
i = 0
```

```
nums[i] = -2
```

Step 3

Choose

```
j = 1
```

`nums[j] = -1`

Remaining pointers

`k = 2`

`l = 5`

Current sum

$-2 + (-1) + 0 + 2 = -1$

Less than target

Move

`k++`

Now

`k = 3`

Current sum

$-2 + (-1) + 0 + 2 = -1$

Still less

Move

`k++`

Now

`k = 4`

Current sum

$-2 + (-1) + 1 + 2 = 0$

Valid quadruplet

`[-2, -1, 1, 2]`

Store it.

Continue searching.

Another valid quadruplet found:

`[-2, 0, 0, 2]`

Later,

`[-1, 0, 0, 1]`

Final Answer

```
[
  [-2, -1, 1, 2],
  [-2, 0, 0, 2],
  [-1, 0, 0, 1]
]
```

Correctness

- Sorting allows efficient use of the two-pointer technique.
 - The outer loops fix the first two numbers.
 - The two pointers efficiently search for the remaining two numbers.
 - Duplicate values are skipped at every level, ensuring only unique quadruplets are added.
 - Therefore, every valid quadruplet is found exactly once.
-

Complexity Analysis

Time Complexity

Sorting:

$O(n \log n)$

Outer loop:

$O(n)$

Second loop:

$O(n)$

Two-pointer traversal:

$O(n)$

Overall:

$O(n^3)$

Space Complexity

Ignoring the output list,

$O(1)$

Extra space is only used for pointers and variables.

Key Points

- Extension of the **3Sum** problem.
- Uses **Sorting + Two Pointers**.
- Duplicate quadruplets are avoided by skipping repeated values of **i**, **j**, **k**, and **l**.
- **long** is used to avoid integer overflow while calculating the sum.
- **Time Complexity:** $O(n^3)$
- **Space Complexity:** $O(1)$ (excluding the result list).

Sort Colors (Dutch National Flag Algorithm)

Problem Statement

Given an array **nums** containing only 0s, 1s, and 2s, sort the array **in-place** so that all 0s come first, followed by all 1s, and then all 2s.

Example

Input

`nums = [2, 0, 2, 1, 1, 0]`

Output

`[0, 0, 1, 1, 2, 2]`

Approach

Since the array contains only three distinct values (0, 1, and 2), we can sort it in a **single traversal** using the **Dutch National Flag Algorithm**.

The idea is to divide the array into four regions:

0s | 1s | Unknown | 2s

Initially, the entire array belongs to the **Unknown** region.

As we scan the array, elements are moved into their correct regions by swapping.

Algorithm

Maintain three pointers:

- **i (Low Pointer):** Indicates the position where the next 0 should be placed.
- **k (Current Pointer):** Traverses the array.
- **j (High Pointer):** Indicates the position where the next 2 should be placed.

Initially,

```
i = 0  
k = 0  
j = n - 1
```

Case 1: Current element is 0

If

```
nums[k] == 0
```

- Swap `nums[i]` and `nums[k]`.
- Increment both `i` and `k`.

Reason:

- The 0 has reached its correct position.
 - Everything before `i` already contains only 0s.
 - Since the swapped element came from the left side, it has already been processed.
-

Case 2: Current element is 1

If

```
nums[k] == 1
```

Simply move to the next element.

```
k++
```

Reason:

- 1 already belongs in the middle region.
 - No swapping is required.
-

Case 3: Current element is 2

If

`nums[k] == 2`

- Swap `nums[k]` and `nums[j]`.
- Decrement `j`.

Do **not** increment `k`.

Reason:

The element swapped from the end has not yet been examined. Therefore, it must be checked in the next iteration.

Dry Run

Input

`nums = [2, 0, 2, 1, 1, 0]`

Initially,

`i = 0`

`k = 0`

`j = 5`

Step 1

`nums[k] = 2`

Swap `nums[0]` and `nums[5]`

Array:

`[0, 0, 2, 1, 1, 2]`

`j = 4`

`k` remains 0

Step 2

`nums[k] = 0`

Swap `nums[0]` and `nums[0]`

Array:

`[0, 0, 2, 1, 1, 2]`

```
i = 1  
k = 1
```

Step 3

```
nums[k] = 0
```

```
Swap nums[1] and nums[1]
```

Array:

```
[0, 0, 2, 1, 1, 2]
```

```
i = 2  
k = 2
```

Step 4

```
nums[k] = 2
```

```
Swap nums[2] and nums[4]
```

Array:

```
[0, 0, 1, 1, 2, 2]
```

```
j = 3  
k remains 2
```

Step 5

```
nums[k] = 1
```

```
k = 3
```

Step 6

```
nums[k] = 1
```

```
k = 4
```

Now,

```
k > j
```

Loop terminates.

Final array:

```
[0, 0, 1, 1, 2, 2]
```

Why is k not incremented after swapping with j ?

Consider:

$[2, 1, 0]$

Initially,

$k = 0$

$j = 2$

Swap

$2 \leftrightarrow 0$

Array becomes

$[0, 1, 2]$

The element that came from index j (which is 0) has never been processed.

If we increment k immediately, we would skip checking this element.

Therefore,

- After swapping with j , decrement only j .
 - Keep k at the same position so the new element can be examined.
-

Correctness

The algorithm maintains the following invariant throughout execution:

- Indices 0 to $i - 1$ contain only 0 s.
- Indices i to $k - 1$ contain only 1 s.
- Indices k to j are unprocessed.
- Indices $j + 1$ to $n - 1$ contain only 2 s.

At every iteration, one element is moved from the unknown region into its correct region while preserving these invariants.

When $k > j$, there are no unprocessed elements left, so the entire array is sorted.

Complexity Analysis

Time Complexity: $O(n)$

- Each element is processed at most once.
- The array is traversed in a single pass.

Space Complexity: $O(1)$

- Sorting is performed in-place.
 - Only a few pointer variables are used.
-

Key Points

- Uses the **Dutch National Flag Algorithm**.
- Maintains three pointers (i , k , j).
- Sorts the array in a **single traversal**.
- Performs sorting **in-place** without using extra memory.
- k is **not incremented** after swapping with j because the swapped element is still unprocessed.
- This is the optimal solution with $O(n)$ time and $O(1)$ space complexity.

Find Kth Largest Element in an Array (Min-Heap Approach)

Problem Statement

Given an integer array `nums` and an integer k , return the **kth largest element** in the array.

Note: The k th largest element is the k th element in the sorted order, **not** the k th distinct element.

Approach

A **Min-Heap (Priority Queue)** of size k is used to efficiently keep track of the k largest elements seen so far.

The idea is:

- Insert every element into the min-heap.
 - If the heap size becomes greater than k , remove the smallest element.
 - After processing all elements, the heap contains exactly the k largest elements.
 - The root of the min-heap is the **kth largest element**.
-

Why Min-Heap?

A Min-Heap always keeps the **smallest element at the top**.

When the heap size exceeds k , removing the smallest element ensures that only the largest k elements remain in the heap.

Thus, after processing the entire array:

- Heap size = k
 - Heap contains the k largest elements.
 - The smallest among these k elements is the **k th largest element**.
-

Algorithm

Step 1: Create a Min-Heap

Create an empty priority queue.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

Step 2: Traverse the Array

For every element in the array:

- Insert the element into the heap.
- If the heap size becomes greater than k , remove the smallest element using `poll()`.

This guarantees that the heap never stores more than k elements.

Step 3: Return the Answer

After all elements have been processed:

- The heap contains the k largest elements.
 - The root of the heap (`peek()`) is the k th largest element.
-

Dry Run

Example

Input

```
nums = [3, 2, 1, 5, 6, 4]
k = 2
```

Execution

Element	Heap After Insertion	Heap Size	Action
3	[3]	1	Keep
2	[2,3]	2	Keep
1	[1,3,2]	3	Remove 1 → [2,3]
5	[2,3,5]	3	Remove 2 → [3,5]
6	[3,5,6]	3	Remove 3 → [5,6]
4	[4,6,5]	3	Remove 4 → [5,6]

Final Heap:

[5, 6]

The smallest element in the heap is:

5

Therefore,

Answer = 5

Another Example

Input

nums = [3, 2, 3, 1, 2, 4, 5, 5, 6]
k = 4

Heap Evolution

Insert 3 → [3]

Insert 2 → [2, 3]

Insert 3 → [2, 3, 3]

Insert 1 → [1, 2, 3, 3]

Insert 2 → [1, 2, 3, 3, 2]

Remove 1

Heap = [2, 2, 3, 3]

Insert 4

Remove 2

Heap = [2, 3, 3, 4]

Insert 5

Remove 2

Heap = [3, 4, 3, 5]

Insert 5

Remove 3

Heap = [3, 4, 5, 5]

Insert 6

Remove 3

Final Heap = [4, 5, 5, 6]

Top of the heap:

4

Therefore,

4th Largest Element = 4

Correctness

- The heap never stores more than k elements.
 - Whenever the heap size exceeds k , the smallest element is removed.
 - Therefore, after processing all elements, only the largest k elements remain.
 - Since the heap is a Min-Heap, its root is the smallest among these k elements.
 - Hence, the root is exactly the **k th largest element**.
-

Complexity Analysis

Time Complexity

- Inserting each element into the heap takes $O(\log k)$.
- There are n elements.

Overall Time Complexity:

$O(n \log k)$

Space Complexity

The heap stores at most k elements.

Space Complexity: $O(k)$

Key Points

- Uses a **Min-Heap (Priority Queue)**.
- Heap size is always maintained as k .

- Whenever the heap exceeds size k , remove the smallest element.
- After processing the array, the heap contains the k largest elements.
- The root of the heap is the **k th largest element**.
- More efficient than sorting the entire array ($O(n \log n)$), especially when k is much smaller than n .

Find Kth Largest Element in an Array (Quick Select Algorithm)

Problem Statement

Given an integer array `nums` and an integer k , return the **k th largest element** in the array.

Note: The k th largest element is the k th element in the sorted order, **not** the k th distinct element.

Approach (Quick Select)

The Quick Select algorithm is an optimized version of Quick Sort. Unlike Quick Sort, which recursively sorts both partitions, Quick Select only processes the partition that contains the required element.

The idea is to repeatedly partition the array around a pivot until the pivot reaches the position of the required element.

Key Observation

If the array is sorted in **ascending order**,

Index :	0	1	2	3	4	5
Value :	1	2	3	4	5	6

- Largest element $\rightarrow$ Index $n - 1$
- 2nd Largest $\rightarrow$ Index $n - 2$
- 3rd Largest $\rightarrow$ Index $n - 3$

Therefore,

Target Index = $n - k$

where n is the length of the array.

Algorithm

Step 1: Find the Target Index

```
target = nums.length - k
```

This converts the kth largest problem into finding the element at the target index in a sorted array.

Step 2: Partition the Array

Choose the **last element** as the pivot.

Maintain two pointers:

- `i` → position where the next smaller element should be placed.
- `j` → traverses the array.

For every element:

- If `nums[j] <= pivot`
 - Swap `nums[i]` and `nums[j]`
 - Increment `i`

After the loop, place the pivot at its correct sorted position by swapping `nums[i]` with the pivot.

The partition function returns the pivot's final index.

Step 3: Compare Pivot Index with Target

There are three possibilities:

Case 1: Pivot Index == Target

The pivot is exactly the required kth largest element.

Return `nums[pivotIndex]`.

Case 2: Pivot Index < Target

The required element lies in the **right partition**.

Update

```
left = pivotIndex + 1
```

Case 3: Pivot Index > Target

The required element lies in the **left partition**.

Update

```
right = pivotIndex - 1
```

Repeat the process until the pivot reaches the target index.

Partition Example

Consider

```
nums = [3, 2, 1, 5, 6, 4]  
Pivot = 4
```

Initial

```
3  2  1  5  6  4  
                ↑ Pivot
```

Traverse the array.

- $3 \leq 4 \rightarrow$ Keep
- $2 \leq 4 \rightarrow$ Keep
- $1 \leq 4 \rightarrow$ Keep
- $5 > 4 \rightarrow$ Ignore
- $6 > 4 \rightarrow$ Ignore

Swap pivot with first larger element.

Before swap

```
3  2  1  5  6  4  
      ↑      ↑
```

After swap

```
3  2  1  4  6  5  
      ↑  
Pivot Index = 3
```

Now,

- Left side contains elements ≤ 4
- Right side contains elements > 4

The pivot is now in its final sorted position.

Dry Run

Input

nums = [3, 2, 1, 5, 6, 4]
k = 2

Step 1

n = 6

Target = 6 - 2 = 4

We need the element at index **4** after partitioning.

First Partition

Pivot = 4

After partition

[3, 2, 1, 4, 6, 5]

Pivot Index = 3

Since

3 < 4

Search only the right half.

left = 4

Second Partition

Remaining array

[6, 5]

Pivot = 5

After partition

[5, 6]

Pivot Index = 4

Since

Pivot Index == Target

Answer

5

Correctness

- The partition operation places the pivot exactly where it would appear in the fully sorted array.
- If the pivot reaches the target index, the required element has been found.
- Otherwise, only one partition can contain the target element, reducing the search space.
- This process continues until the target index is reached.

Hence, the algorithm always returns the kth largest element.

Complexity Analysis

Average Case

Time Complexity: $O(n)$

- Each partition discards approximately half of the remaining elements on average.
-

Worst Case

Time Complexity: $O(n^2)$

Occurs when the pivot is consistently the smallest or largest element (e.g., already sorted array with poor pivot selection).

Space Complexity

$O(1)$

- The algorithm is performed in-place.
 - No additional arrays are used.
-

Why Quick Select is Faster than Sorting

Sorting the entire array requires

$O(n \log n)$

However, Quick Select only explores the partition containing the required element.

Therefore,

- Sorting $\rightarrow$ Entire array

- Quick Select → Only one partition

This reduces the average time complexity to **$O(n)$** .

Key Points

- Uses the **Quick Select** algorithm.
- Partition logic is identical to Quick Sort.
- Only one side of the partition is processed after each iteration.
- The target index is calculated as $n - k$.
- Average Time Complexity: **$O(n)$**
- Worst Case Time Complexity: **$O(n^2)$**
- Space Complexity: **$O(1)$**
- More efficient than sorting the entire array when only the k th largest element is required.

Minimum Size Subarray Sum (Sliding Window Approach)

Problem Statement

Given an array of positive integers `nums` and a positive integer `target`, return the **minimum length of a contiguous subarray** whose sum is **greater than or equal to** `target`.

If no such subarray exists, return 0.

Approach

Since all elements in the array are **positive**, we can efficiently solve the problem using the **Sliding Window Technique**.

The main idea is to maintain a window whose sum is tracked continuously.

- Expand the window by moving the right pointer until the sum becomes greater than or equal to the target.
 - Once the condition is satisfied, shrink the window from the left as much as possible while maintaining the condition.
 - During this process, keep updating the minimum window length.
-

Algorithm

Step 1: Initialize Variables

- `i` → Left pointer of the sliding window.
 - `j` → Right pointer of the sliding window.
 - `sum` → Stores the current window sum.
 - `length` → Stores the minimum valid window length. Initialize it to `Integer.MAX_VALUE`.
-

Step 2: Expand the Window

Move the right pointer (`j`) one step at a time.

For every element,

- Add `nums[j]` to `sum`.

```
sum += nums[j]
```

Step 3: Shrink the Window

Whenever

```
sum >= target
```

the current window satisfies the condition.

Now,

1. Calculate the current window size.

```
size = j - i + 1
```

2. Update the minimum length if the current window is smaller.
3. Remove the leftmost element from the window.

```
sum -= nums[i]
```

4. Move the left pointer one step forward.

```
i++
```

Repeat this process until the window sum becomes smaller than the target.

Step 4: Continue Expanding

Move the right pointer again and repeat the above process until the end of the array is reached.

Step 5: Return the Answer

- If `length` is still `Integer.MAX_VALUE`, no valid subarray exists, so return 0.
 - Otherwise, return `length`.
-

Dry Run

Example

Input

`target = 7`

`nums = [2,3,1,2,4,3]`

Iteration 1

`Window = [2]`

`Sum = 2`

`Sum < 7`

Expand window.

Iteration 2

`Window = [2,3]`

`Sum = 5`

Expand window.

Iteration 3

`Window = [2,3,1]`

`Sum = 6`

Expand window.

Iteration 4

`Window = [2,3,1,2]`

Sum = 8

Sum >= 7

Window Length = 4

Minimum Length = 4

Shrink the window.

Remove 2

Window = [3,1,2]

Sum = 6

Iteration 5

Add 4.

Window = [3,1,2,4]

Sum = 10

Length = 4

Shrink

Remove 3.

Window = [1,2,4]

Sum = 7

Length = 3

Minimum Length = 3

Shrink again.

Remove 1

Window = [2,4]

Sum = 6

Iteration 6

Add 3.

Window = [2,4,3]

Sum = 9

Length = 3

Shrink.

Remove 2

Window = [4,3]

Sum = 7

Length = 2

Minimum Length = 2

Shrink again.

Remove 4

Window = [3]

Sum = 3

End of array.

Output

2

The smallest subarray is

[4,3]

whose sum is 7.

Correctness

- The window always represents a **contiguous subarray**.
 - Since all numbers are positive, expanding the window always increases (or maintains) the sum, while shrinking the window always decreases it.
 - Every possible valid window is considered exactly once.
 - Whenever the window satisfies the target condition, its length is compared with the current minimum.
 - Therefore, the algorithm correctly returns the minimum length of a contiguous subarray whose sum is at least the target.
-

Complexity Analysis

Time Complexity: $O(n)$

- Each element is added to the window once by the right pointer.

- Each element is removed from the window at most once by the left pointer.
- Hence, both pointers traverse the array only once.

Space Complexity: $O(1)$

- Only a few variables are used regardless of the input size.
-

Key Points

- Uses the **Sliding Window Technique**.
- Works because **all array elements are positive**.
- Maintains a running sum instead of recomputing it.
- Expands the window to satisfy the target and shrinks it to find the smallest valid window.
- Much more efficient than the brute-force approach ($O(n^2)$), achieving **$O(n)$** time complexity.